

MAdES — a worked example

This document is an example of a MAdES signature, and it is also the thing being signed. Everything described below happened to this file.

If you are reading it in a Markdown viewer, you are seeing the document. If you open it in a text editor, you will find a block at the bottom that the viewer hides. That block is the signature.

What is about to happen to this file

A signature is not applied to a document so much as *fixed around* it. Four things happen, in this order, and the order is not ours to choose.

1. **The content is frozen.** The text above and below this line is canonicalised — line endings normalised, trailing whitespace stripped — so that two systems reading the same document agree on the same bytes. From this point on, changing a single character changes what was signed.
2. **A certificate is issued.** It carries the signer's name, their country, and a stable identifier for them. It lives for minutes, not years. That sounds fragile and is the opposite: a key that exists briefly is a key that cannot be stolen later.
3. **The signer confirms.** Not here — on the signing service's own page, with a code sent to their own mailbox. The document is never in a position to claim that someone agreed. It can only ask, and be told.
4. **The signature is bound to these exact bytes.** The authorisation covers this document's digest and no other. Presenting it for anything else is refused.

What you will find below the text

A block: an HTML comment whose opening marker is `maDES-sig`. Every conforming Markdown renderer hides it, and every text editor shows it. Invisible in view, present in source, which is where a signature belongs.

A note on this paragraph. You will notice that this document never writes the opening marker in full. It cannot: a verifier locates blocks by scanning for that exact sequence of bytes, and it does not know the difference between a signature and a sentence describing one. Spelling it out here would create a second, empty block — and a reader would be told this document's signature was broken when it is perfectly sound.

That is a real limitation of the format as it stands, and it is worth stating plainly in the one document most likely to trip over it: **a specification cannot presently be signed with the thing it specifies**. A false failure on a valid signature costs exactly what a false success costs, and this is the narrow case where the format still produces one.

Earlier drafts of this format used a fenced code block instead. That renders, and readers opening a signed file saw kilobytes of base64 where their document should have been. The comment was not a clever trick; it was an apology.

Inside the block:

- **signer** and **signer-kind** — who signed, and *what* signed. A human or a machine, stated in a field that falls under the signature and mirrored by the certificate, so the two can be held against each other. Markdown is the format where people and programs take turns; this is the question it can answer that PDF never had to ask.
- **commitment** — what the signature means. Producing a document and agreeing to one are different acts, and they are recorded differently.
- **certificate-chain** — the certificate travels inside the file. Nothing has to be fetched, so nothing can fail to be fetched in five years.
- **timestamp** — an RFC 3161 token proving the signature existed while the certificate was still valid. This is what lets a short-lived key produce a long-lived signature. An expired certificate on a PAdES signature is normal, and is not a finding.
- **signature** — the bytes themselves.

There is also an **appearance** field, and it deserves a word. When this document is signed, a visible seal is added to it: a small panel with the signer's name and the moment of signing. That seal is part of what gets signed — not decoration applied afterwards. What a reader sees and what the signature covers are the same thing, which is the whole point.

How to check this yourself

Take the file. Do not trust the service that produced it, and do not trust the badge in whatever viewer you are using.

1. Strip the signature block and rebuild the signing input from what remains, following the rules in the specification.
2. Verify the signature against the certificate embedded in the file.
3. Check that certificate's chain against a trust anchor you already hold.
4. Check that the timestamp covers the signature and falls inside the certificate's validity window.

If all four hold, this document had exactly this content at the stated moment, and the named signer stood behind it. If any of them fails, you have learned something more useful than a green tick.

The PDF beside this file

The same signature ceremony produces a second artefact: this document rendered to PDF and signed as PAdES, in the same act, with the same certificate and the same confirmation. One authorisation, two representations.

They are not copies of each other. The MAdES signature covers the text; the PAdES signature covers the page. A reader who wants the source gets a file that carries its own proof; a reader who wants a document that opens in Acrobat gets one that validates there. Neither is a rendering of the other's claim.

This document is published as a worked example alongside the MAdES specification. It is signed with a certificate from a test authority, so a verifier that does not hold that root will report a valid signature with an unrecognised anchor — which is the correct answer, and a good illustration of why “untrusted” and “invalid” must never be the same word.